Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Design and Deployment

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Tiered Hardware Architectures
- Deployment Models
- Service types in Cloud

Some feedback has also asked about common distributed application patterns…

- Independent learning section on:
  - Layers revisited
  - Model View Controller (MVC)
  - Model View ViewModel (MVVM)
  - Model View Presenter (MVP)

UNIVERSITY of HULL

Layers and Tiers

- Layer (recall – also more in the reading section at the end)
  - Logical structuring mechanism for elements in a software solution
- Use of layering
  - Gives flexibility in application deployment
    - Layer boundaries ease distribution
    - Not every layer boundary is a good distribution boundary
  - Reduces dependencies; improves testability

- Tier
  - Physical structuring mechanism for system infrastructure
- Use of tiers
  - Allows optimization of computers for specific tasks
  - Allows separation of servers with different security, fault tolerance and scalability requirements
  - Reduces administration and deployment overhead

UNIVERSITY of HULL

Tiers

Layering is concerned with logical, code-based grouping and boundaries

Tiers are concerned with **physical** groupings and boundaries

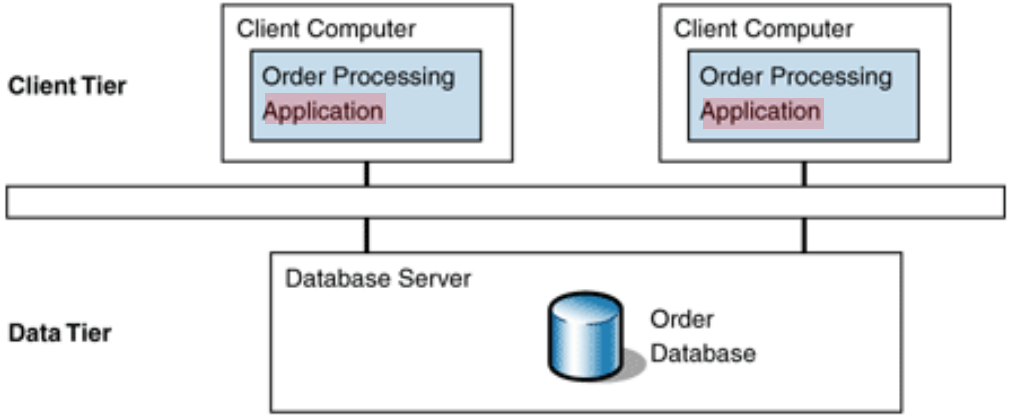
Layers and sublayers may be split across tiers

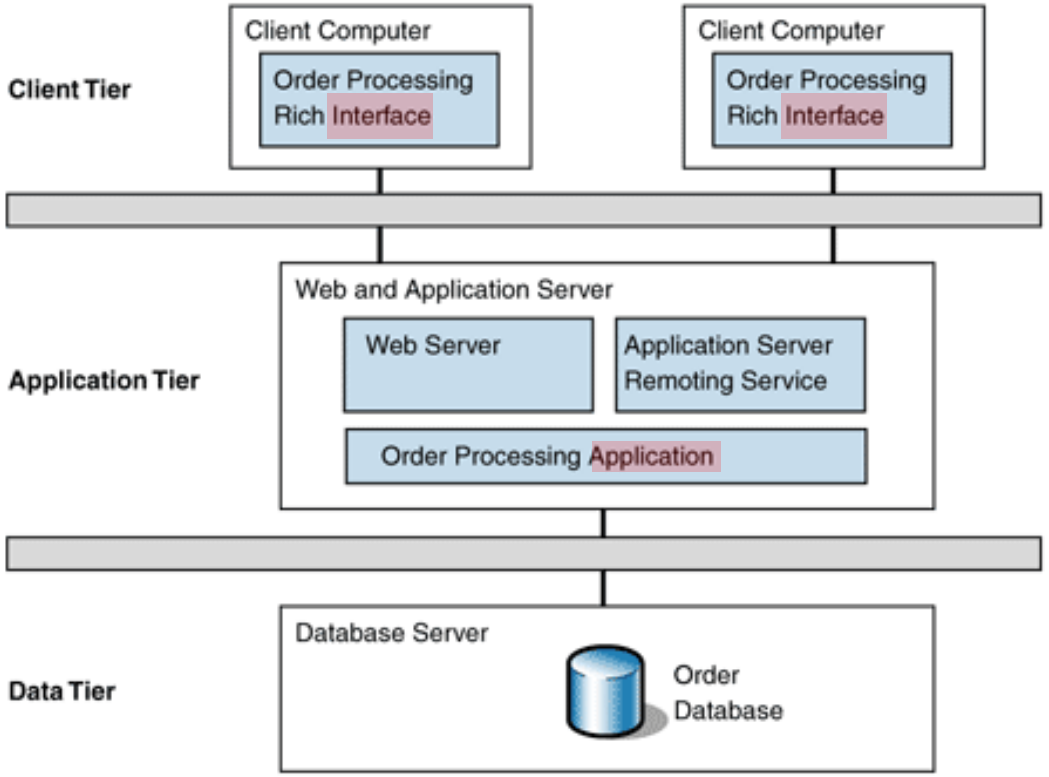
…or exist within a single tier

Tiers consider where the application will run and this has an effect on the:

- Scalability of the system
- Performance of the system
- Availability of the system
- Balance of workload
- Complexity of maintenance/error tracking

Two/Three Tiers





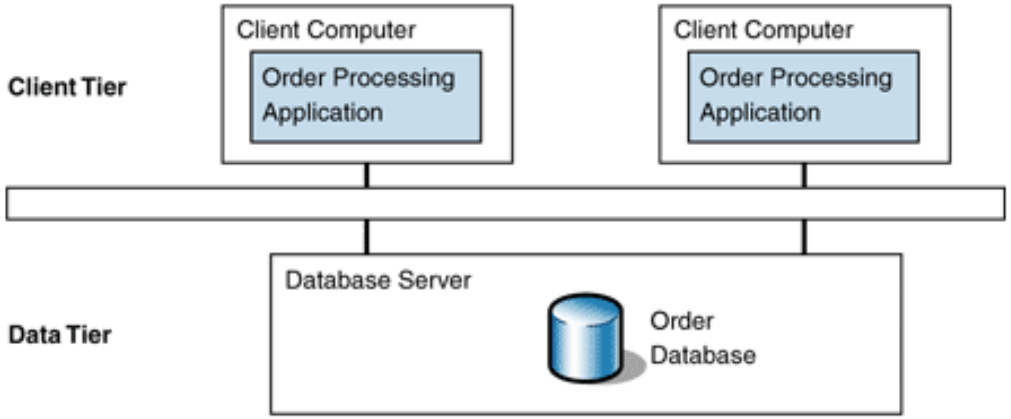
UNIVERSITY of HULL

Example Tiers

- Client tier
  - Workstations, PDAs, browsers. Can be many systems.
  - Scalability depends on connections to application tier
  - Separate security context – may be insecure
- Application tier
  - Business components and web servers
  - Scalable by adding more hardware (server farm)
  - Servers inside DMZ, shared security context
- Data tier
  - Database servers
  - Optimised for I/O throughput and hardware speed
  - Scalability and fault-tolerance means multiple servers (clusters)
  - Clusters may (must?) share state information
  - Most stringent security

Two Tiers – Good?

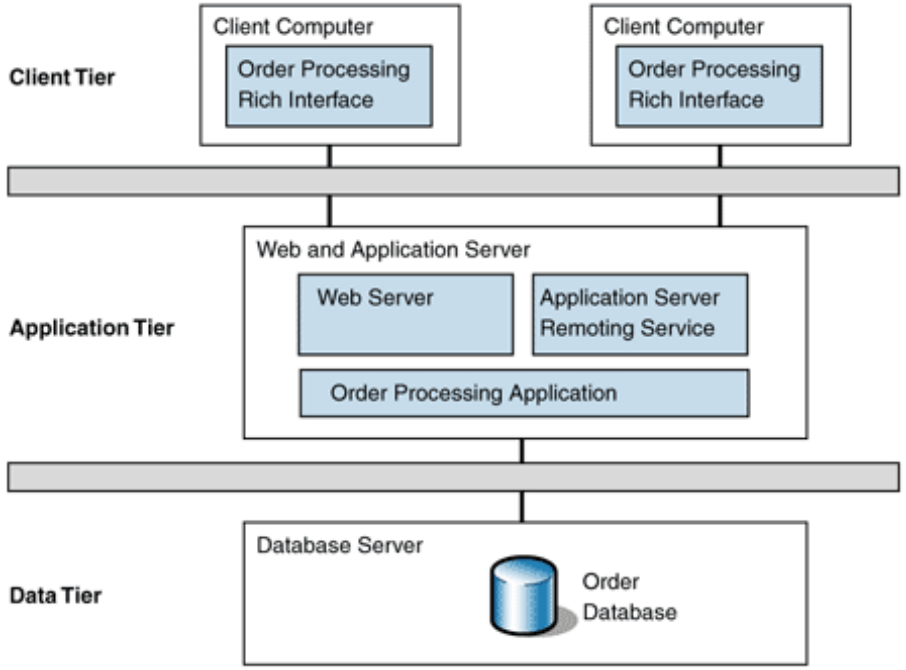
- Total trust in client
  - No filtering or data validation
  - Limited authentication and authorisation
  - Limited logging or non-repudiation
- Difficult for independent clients to manage concurrency
  - No co-client interaction
- Clients must be ‘big’. Capable of doing all of the processing
- Applications can be different



UNIVERSITY of HULL

Three Tiers – Good?

- Three tiers
  - Scalability and fault tolerance
  - Separating the client computer from the application logic supports the development and distribution of thin-client applications (which may be different).
  - Each tier can operate within its own security context.
  - Having the Web server and business components on the same computer increases performance.
  - Concurrent is easier because one server manages concurrent data access.
  - Maintenance may only be necessary on server rather than all clients

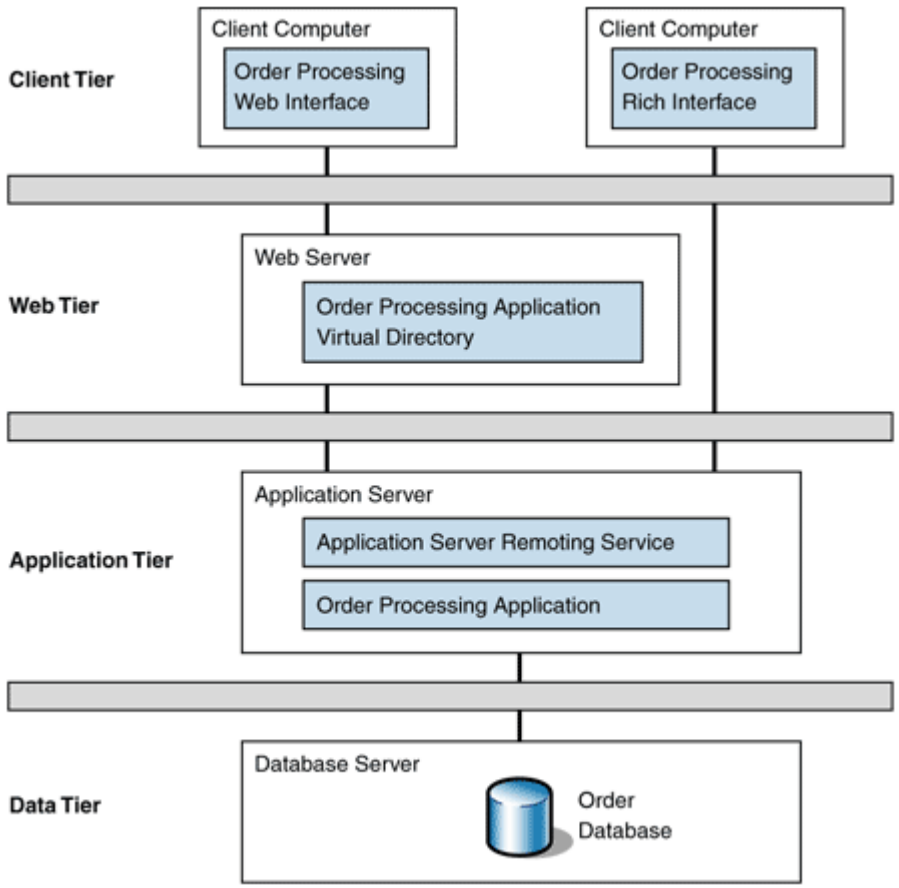


- Four tiers
  - Three tiers good, four tiers better?

UNIVERSITY of HULL

Four Tiers (An example)

- Web servers and application servers separated into own tiers
  - Allows optimization of servers according to resource usage profiles (e.g. Web vs API)
- For scalability and fault-tolerance
  - Web tier uses server farm
  - Application tier uses server cluster
  - Adds cost and complexity
- Web tier security
  - Allows external access outside DMZ without exposing internal data



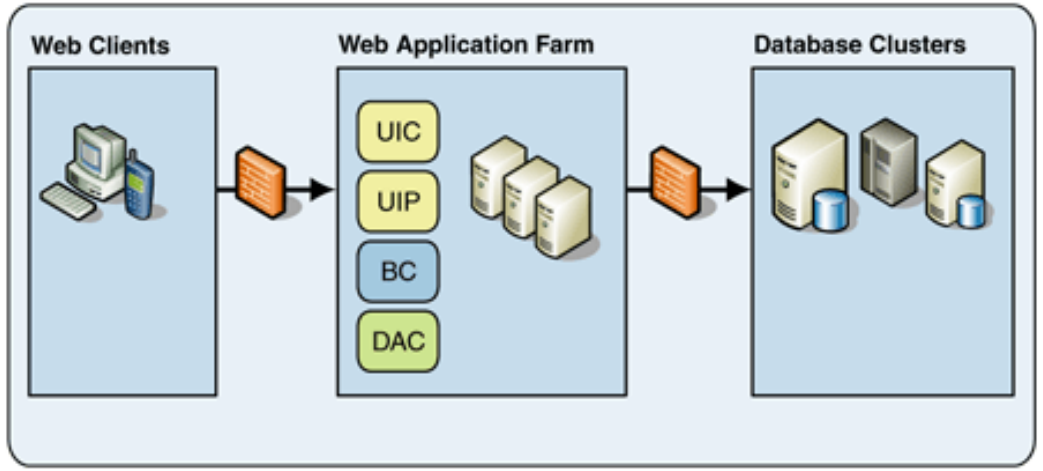
UNIVERSITY of HULL

Deploying to Tiers

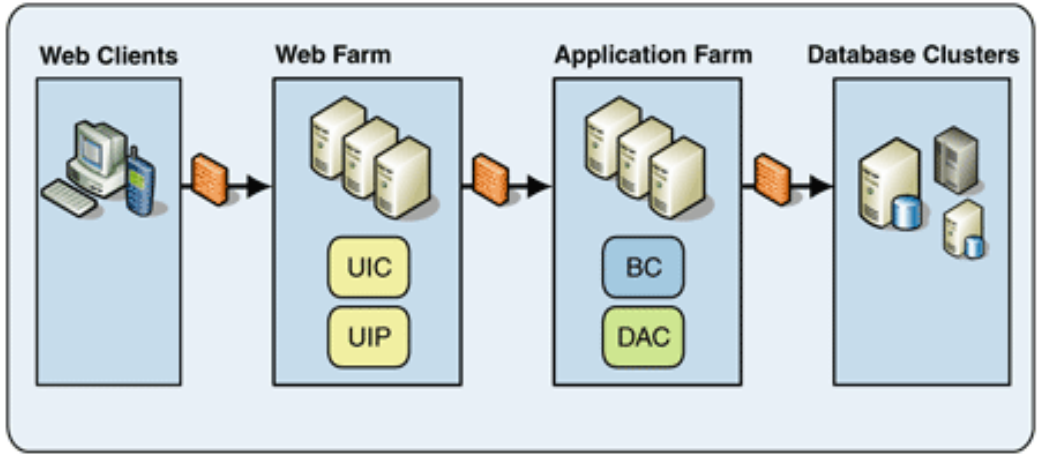
- How should components be deployed to tiers?
  - Direct mapping of layers to tiers often not the best solution
  - Each additional tier adds development effort, complexity and cost
  - Meeting security requirements often results in addition of extra tiers
- Two types of tier
  - Specialized tiers
    - Optimized for hosting components with specific resource requirements
  - General tiers
    - Generically configured servers
- Several common deployment models have been identified
  - Large applications are often hybrids of these

Common Models

- Simple Web Application
  - All components deployed to single general-purpose tier



- Complex Web Application
  - Uses specialized tiers
  - Presentation tier
    - Maximum security
    - Handling many concurrent users in bursts
    - Presentation is less intensive than business logic so more requests can be handled
  - Application and data tiers
    - Steady stream of requests from few sources

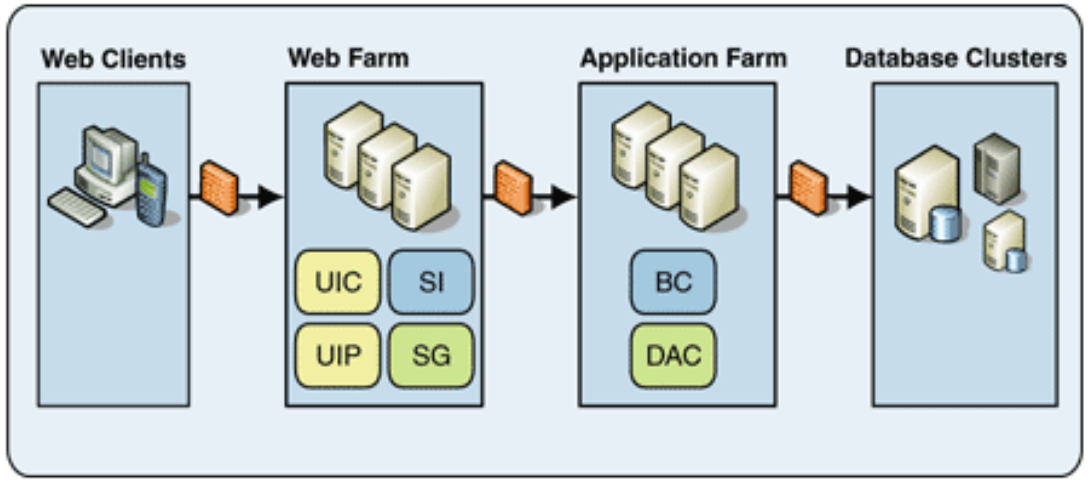


UIC – User Interface Components
UIP – User Interface Process Components
SI – Service Interfaces

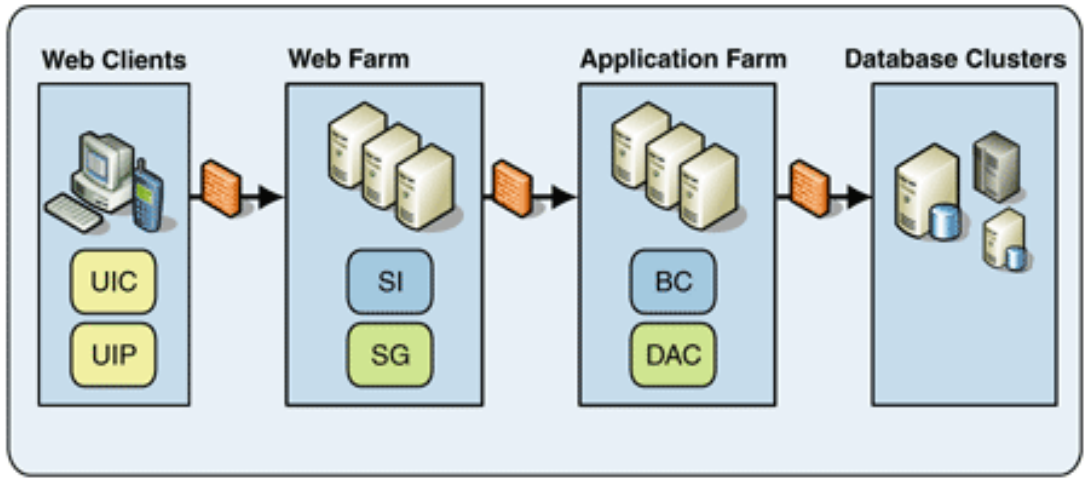
SG – Service gateways
BC – Business Components
DAC – Data Access Components

Common Models

- Extended Enterprise Application
  - Consumes services offered by other applications
  - Makes services available for consumption by others
  - May also offer a web presentation element



- Smart Client Application
  - User interface components deployed in client tier
  - Useful when high degree of user interaction is required
  - Client has more trust and has more application knowledge (e.g. phone app)



UIC – User Interface Components
UIP – User Interface Process Components
SI – Service Interfaces
SG – Service gateways
BC – Business Components
DAC – Data Access Components

Towards the Cloud

- Common Distributed Service Types (Anything as a Service – XaaS):
  - Backend as a Service (BaaS)
  - Software as a Service (SaaS)
  - Infrastructure as a Service (IaaS)
  - Platform as a Service (PaaS)
  - Function as a Service (FaaS)

UNIVERSITY of HULL

Anything as a Service - XaaS

- More and more products/technologies/functions are now offered as a service

- As we know, services are great…
  - Available over the internet
  - Don't require the client to have specific hardware
  - Not necessary install on the client side
  - Maintain loose coupling between components
  - Very difficult to pirate functionality/software exposed as a service
  - Easy to update/maintain without annoying users

- Businesses like services –
  - Rather than requiring everyone to own/run/maintain an application on their device…
    (lots of space used up and processor time wasted)
  - & Rather than requiring everyone to have powerful CPU/GPU and lots of RAM on their device PC…

…why not allow everyone to access the software as a service?

  - This saves energy/time/money as only the server(s) do the work
    and they scale up to the number of concurrent users.

UNIVERSITY of HULL

XaaS in The Cloud

- Usually the XaaS notation is used to describe functionality that has been moved to The Cloud

- There is a misconception that this means Amazon or Google is now hosting it.
- Or that there is some clever server management software balancing deployment and requests across thousands of machines.
  - Actually this isn't necessary – but it is a common implementation
  - The main thing to identify is that the server allows access to a shared pool of resources
  - This may only be one server shared amongst a few people/apps, or could be 1000s
  - XaaS can be deployed to a remote (networked) server – rather than run locally – and it is "in The Cloud"
  - This 'cloud' may be private to a single company or shared between a few collaborators

- We will be discussing The Cloud more as a concept in the next session…

UNIVERSITY of HULL

Function as a Service - FaaS

- Deploy functions (think methods) as a service

- Each function is separate from other functions
  - BUT they may be chained together
  - OR there may be a number of functions within the same ‘system’

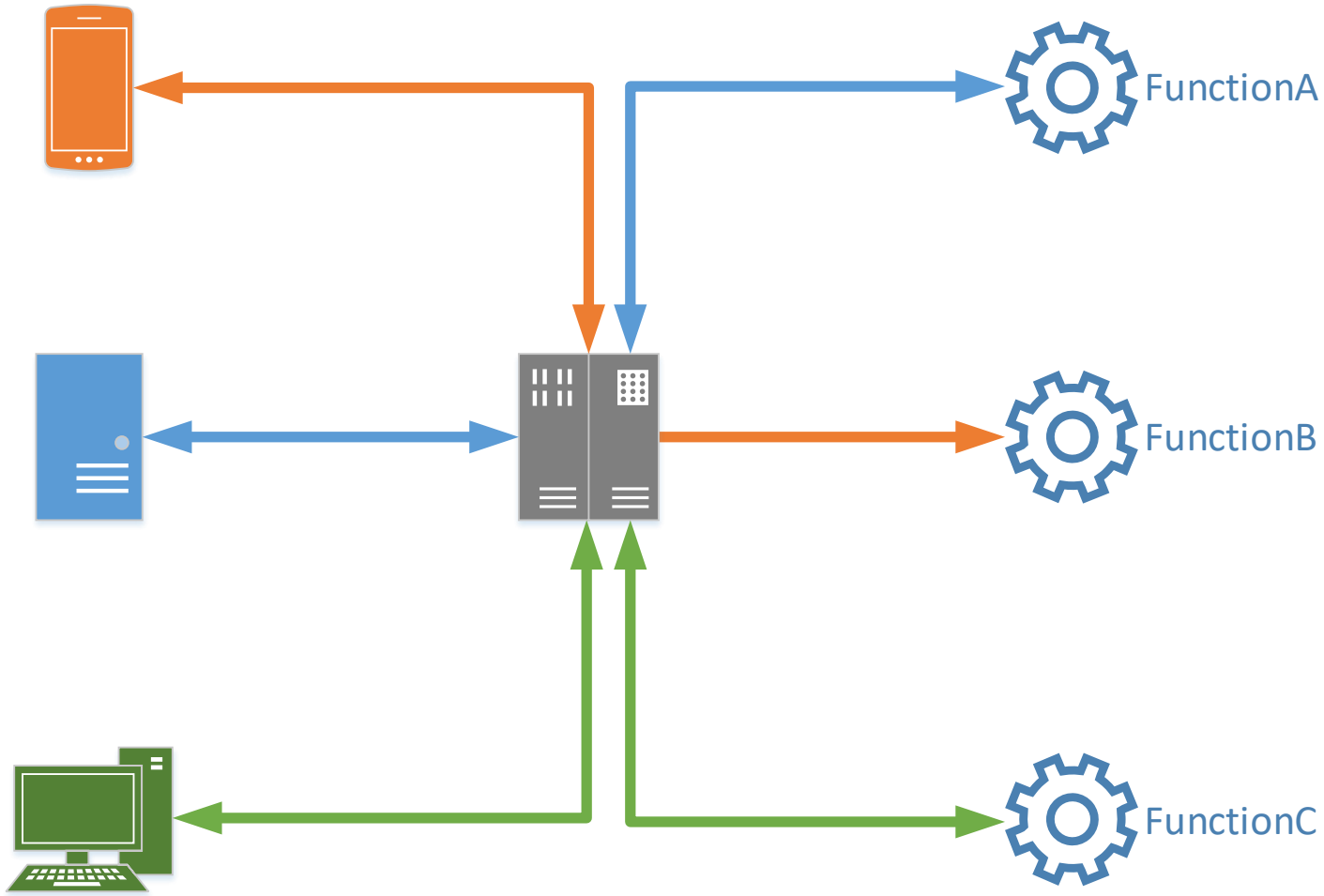
- Typically a listener server waits for requests for particular functions
- When it gets a request it spins up a process to handle it
  - You tell the listener what to do in each situation

- Once the request has been handled the process is thrown away
  - Data cannot be persisted in memory at all

- In theory, a FaaS listener is not coupled to any one application/service
  - Each function is unique and independent

UNIVERSITY of HULL

Function as a Service - FaaS



These could all be the same function or all different.

Processes are spun up independently as requests are made.

UNIVERSITY of HULL

Backend as a Service - BaaS

- A suite of common APIs managed through a hosting and delivery provider
  - Requests are all received by a single service provider – Clients don't need to know where to send different requests
  - Requests are then sent to the correct API(s) for processing

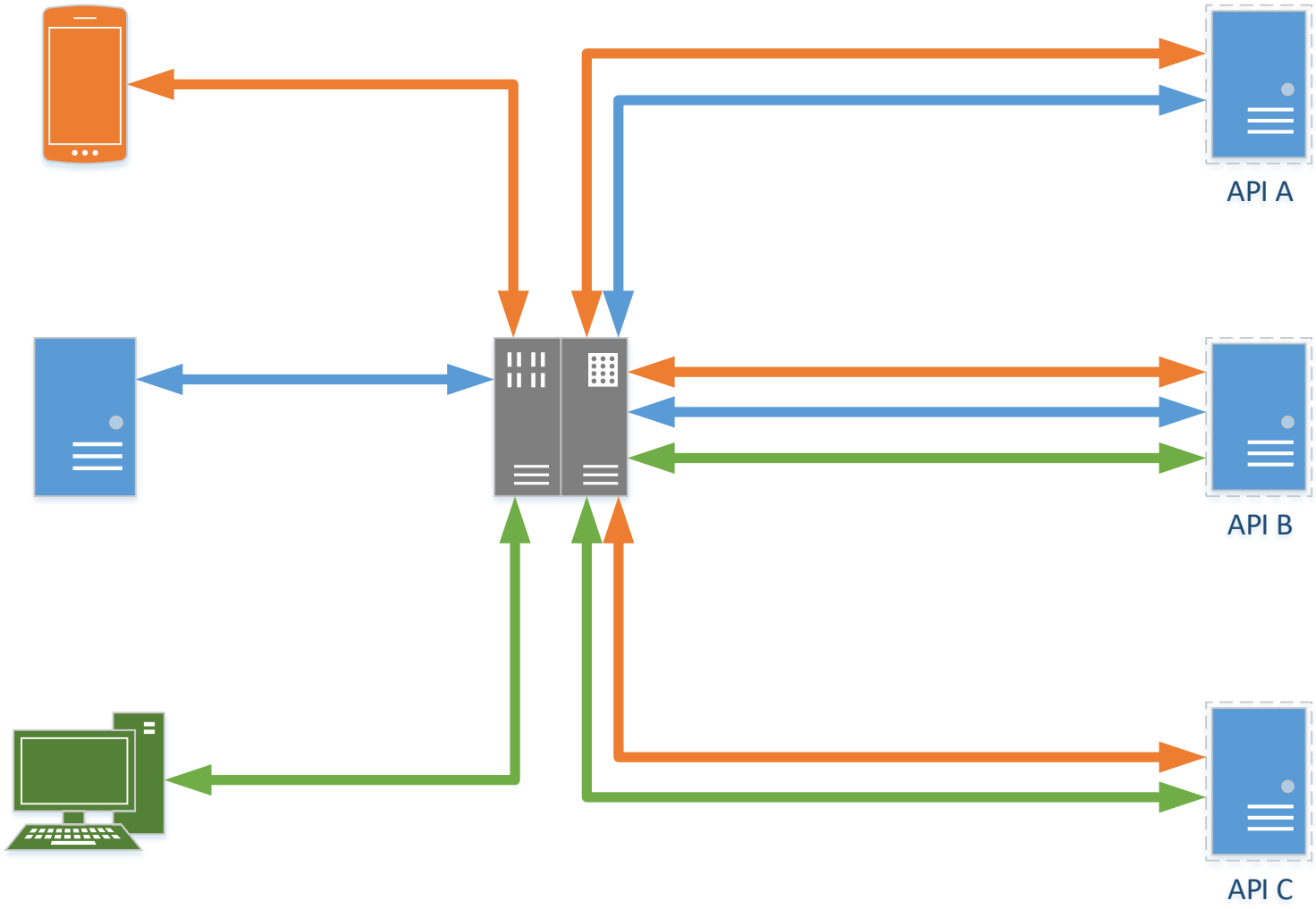
- In similarity with FaaS, a BaaS service manages requests and replies and routes them to the right places
  - However, usually a BaaS system may connect a number of APIs (with multiple functions each)
  - And usually these APIs are 'always on' (perhaps dormant but resources are already allocated)
    - E.g. ASP.Net Web API
      - A new thread is generated each time your API receives a new request
      - However, resources are allocated to your Web API application (unlike FaaS)

- The key part of a BaaS service is that each API is reusable by different clients in different situations
  - Most applications want to use a DB in a similar way, authenticate users, store files, use push notifications, etc.
  - Each API is application agnostic (uncoupled from any one application)
  - Any client can use features of an API connected to the BaaS service once they are set up to do so

- Normally BaaS services are developed BEFORE applications
  - Then any application can be built to consume the service features

UNIVERSITY of HULL

Backend as a Service - BaaS



These APIs are all different.

But single applications may use more than one API.

Clients don't need to know which API they are using for each request.

The BaaS service does the routing.

UNIVERSITY of HULL

DataBase as a Service - DBaaS

- A DBaaS is similar to BaaS except it is specifically for databases
- It allows for live database management/manipulation and stronger security

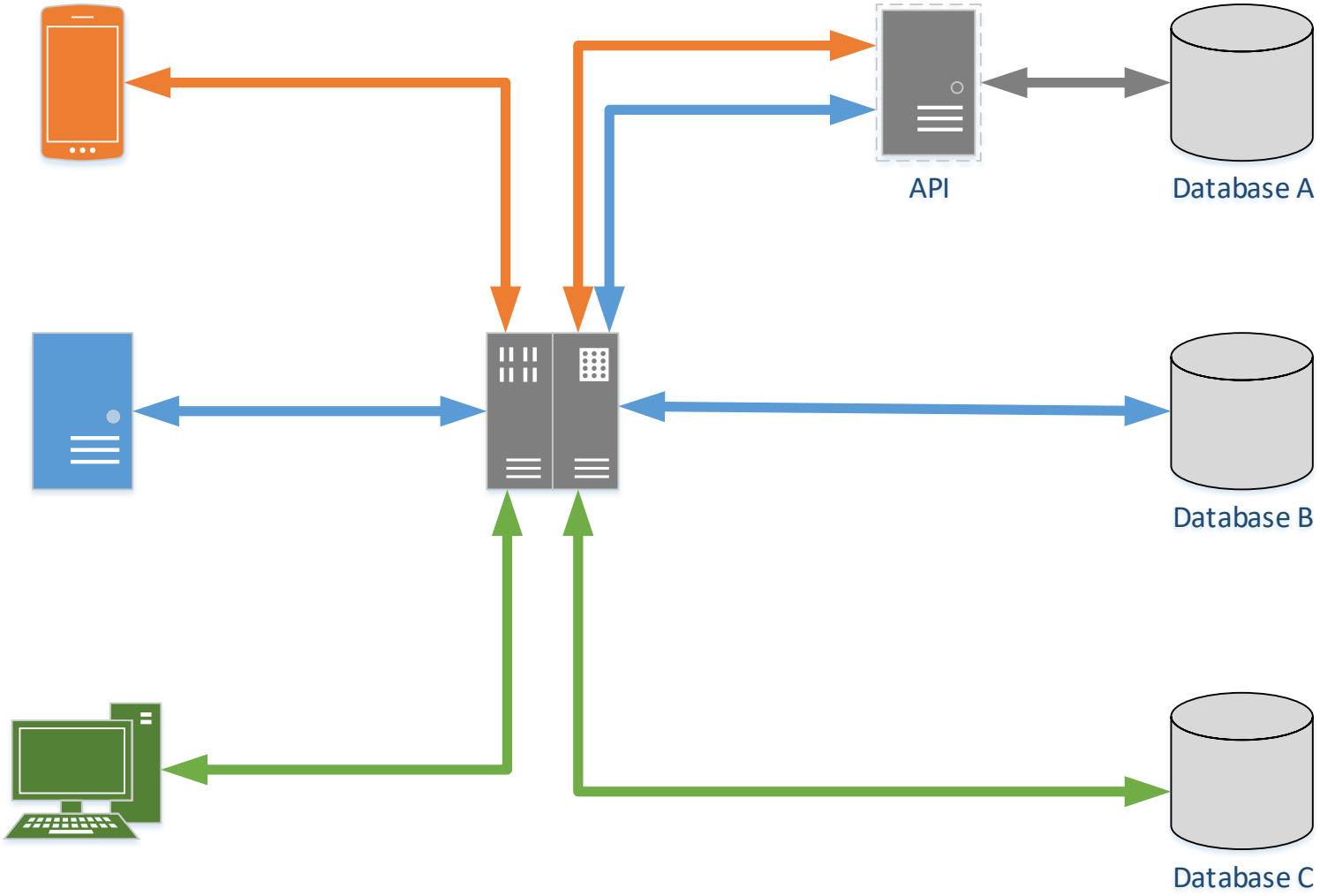
- Databases are usually per-application so usually there is only one application accessing a database
- However, this is not a rule
  - There may be cases where more than one application accesses the same DB

- Management access to the database is provided as a service
  - This means you can easily configure the database programmatically via an API
  - If your app suddenly gets a surge of new users the database(s) can be automatically scaled to manage the load

- Typically application access to the database is managed as a normal DB connection string
  - Although it may also be managed through an API
  - Or through a specific service function SDK

UNIVERSITY of HULL

DataBase as a Service - DBaaS



UNIVERSITY of HULL

Software as a Service - SaaS

- Software as a Service is entire applications which are centrally hosted.

- Software is available on-demand
  - Typically available using a thin client, often via a web browser but not necessarily

- Software is installed on one or more server machines
  - Each client uses a copy of the software by interacting with it via service calls (e.g. RPC or possibly SOAP/REST)
  - Virtual machines may be created automatically to scale to the number of concurrent users

- Client is normally just the UI
- Server does all the work

E.g. Google Docs / Office 365 (online) / Some Mobile Web Apps

Software as a Service - SaaS

- Advantages
  - Very easy to update the entire software application without needing each client to update
  - A single configuration and environment prevents runtime issues based on OS/hardware/local permissions/etc.
  - Easy to deploy a thin client to any device/system/OS with little work
  - No need for users to install/maintain the software or store any data locally
  - You manage the security and access so aren't reliant on another (possibly compromised) device
  - Easy to get error logs and perform general administrative tasks

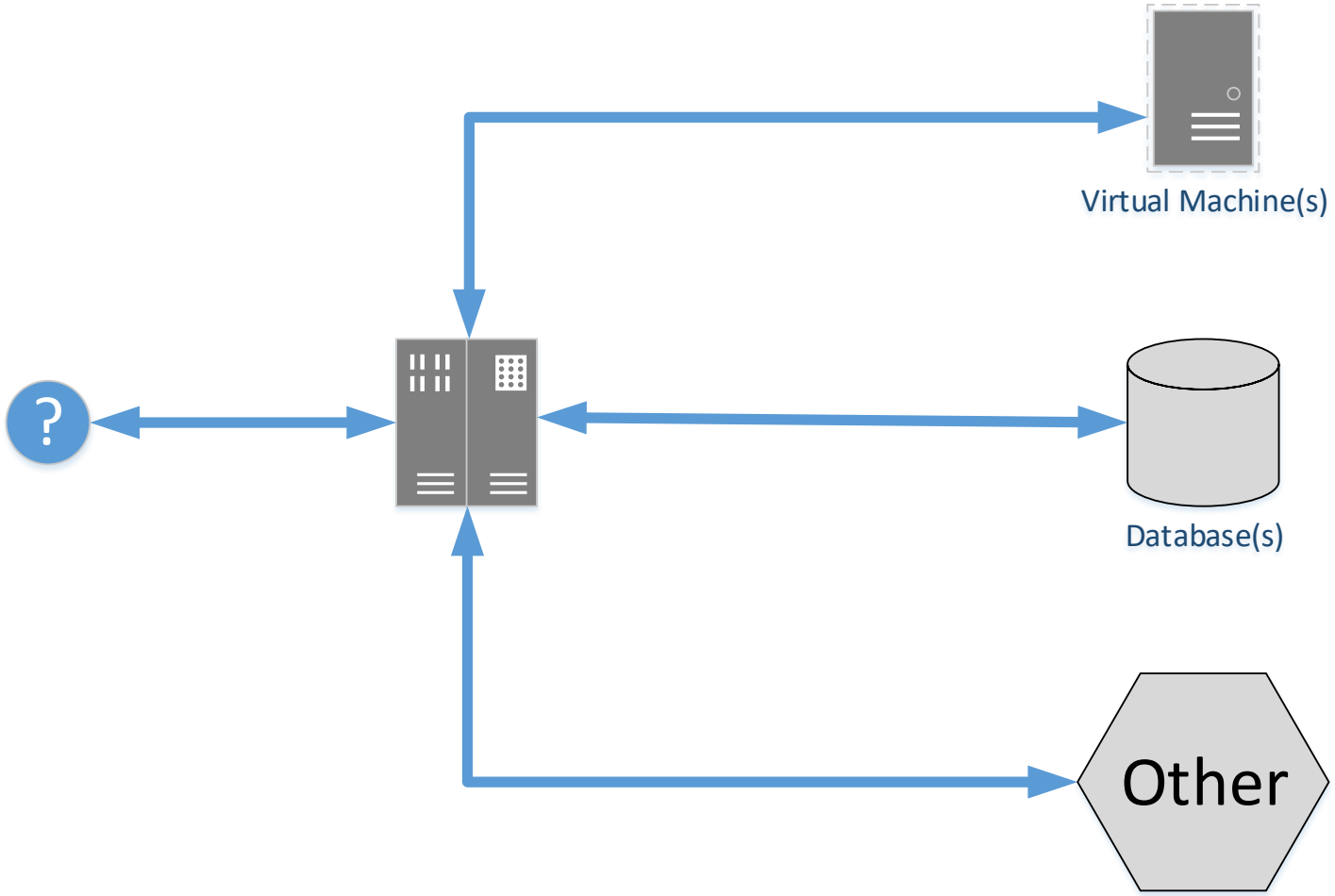
- Disadvantages
  - Servers need to be powerful and have good storage to do all of the application work
  - It is costly to do all of the work for the client
  - If part of the pipeline breaks all clients may lose access
  - The server owner is responsible for any data loss/leak
  - The client must be 'always connected' to make use of the remote application
  - May be difficult to interact with client devices (printer/storage devices/webcam) over the Internet

Infrastructure as a Service - IaaS

- A common provider offering access to infrastructure that the customer has ultimate control over
- The service allows configuration, maintenance and modification of software elements of this infrastructure – but the hardware is managed for you
- You may also be able to scale your infrastructure to manage the number of concurrent users you have or spin up new infrastructure if a server fails/crashes
  - You may need to write applications to do this work for you or do it manually
- Usually, applications can then use the infrastructure
  - Directly (via IP for example) or indirectly (through another service)
- Service provider doesn't mind what you use the infrastructure for
  - But can provide some useful features for you like…
    - Automatic logging
    - Utilisation reporting
    - Common functionality/software/APIs preinstalled or ready to install

UNIVERSITY of HULL

Infrastructure as a Service - IaaS



UNIVERSITY of HULL

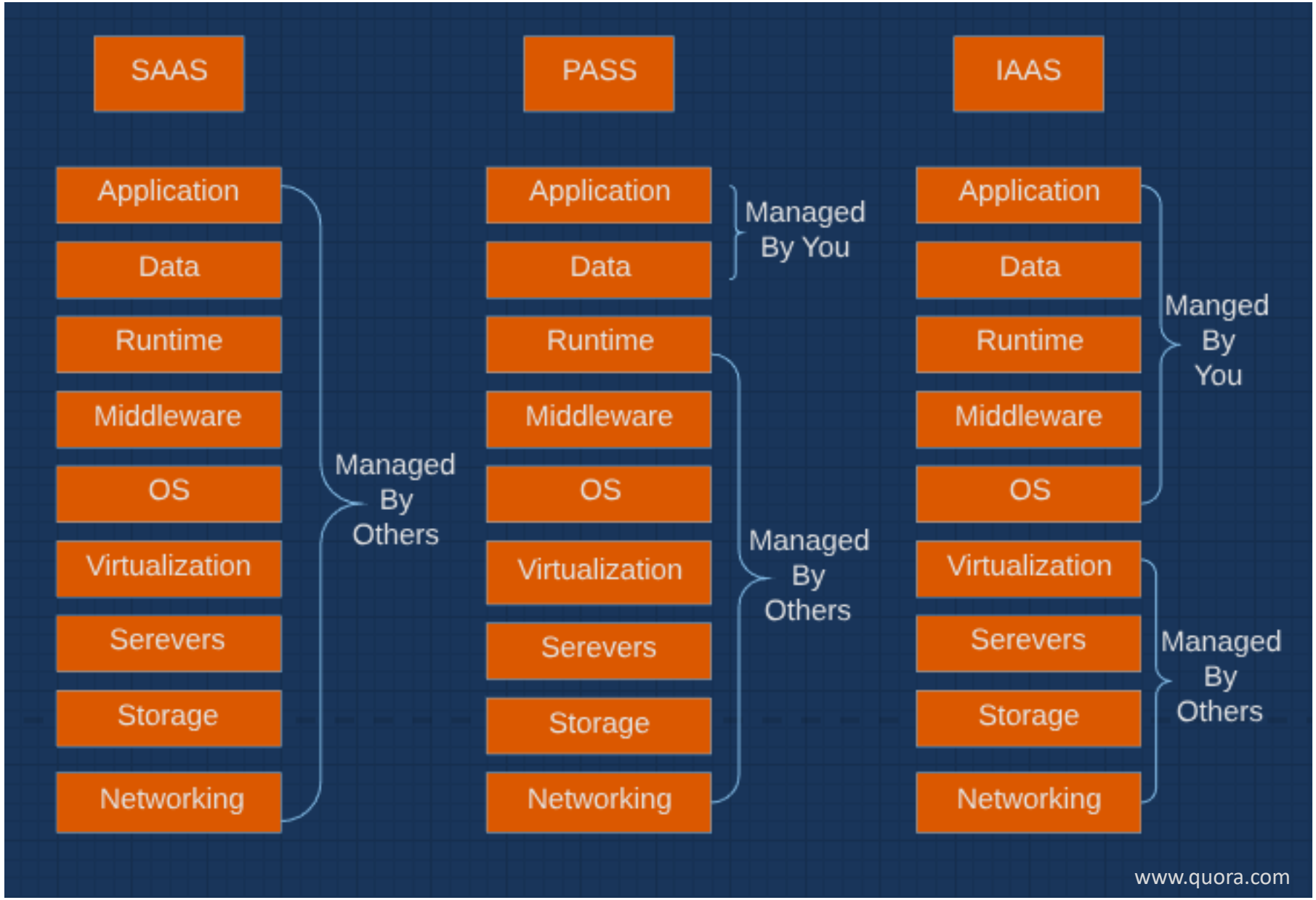
Platform as a Service - PaaS

- Very similar to Infrastructure as a Service but the provider manages the server(s) for you

- This can be ‘smart’ to:
  - Automatically provide scaling
  - Go into a low-power mode when servers/applications aren’t being used
  - Update itself automatically as new versions of software/firmware are released
  - Shut down once predefined limits (money/time) have been reached
  - Restart after failure

- Often, PaaS services allow you to upload an application and the platform will host it for you
  - You may be able to configure it to some degree…
  - …but you have little actual control over the server/infrastructure itself

IaaS vs. PaaS vs. SaaS



Layers

Complications, considerations and HOW?!

Independent Learning

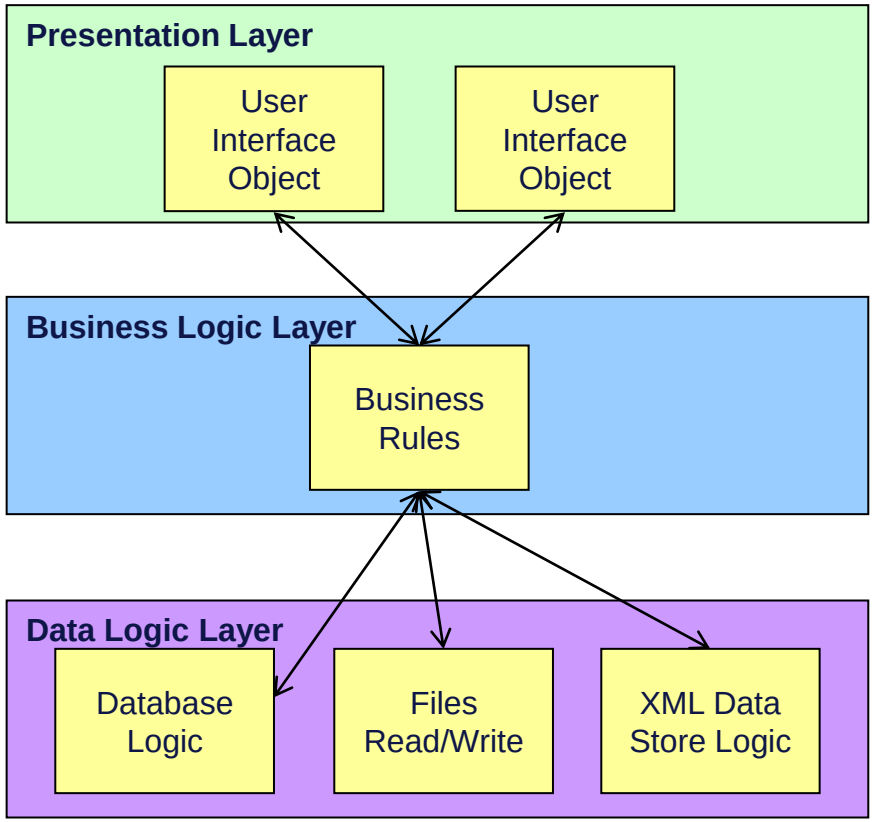
UNIVERSITY of HULL

Problem Area

- Context
  - Large enterprise application, many components
- Problem
  - How to structure for maintainability, reusability, scalability, robustness, security?
- Forces
  - Localising changes; separation of concerns; independent teams; loose coupling
- Solution
  - Layers

Layered Applications

- Layered design is sound software engineering
  - Can be difficult to avoid!
- Layers separate logical groupings of functionality
  - Make functionality easier to change without knock-on side-effects
  - Assist with security (restrictive boundaries)
  - May exist on different devices/processes or in a single process
  - Logical grouping improves component cohesion
  - Each layer may consist of several components



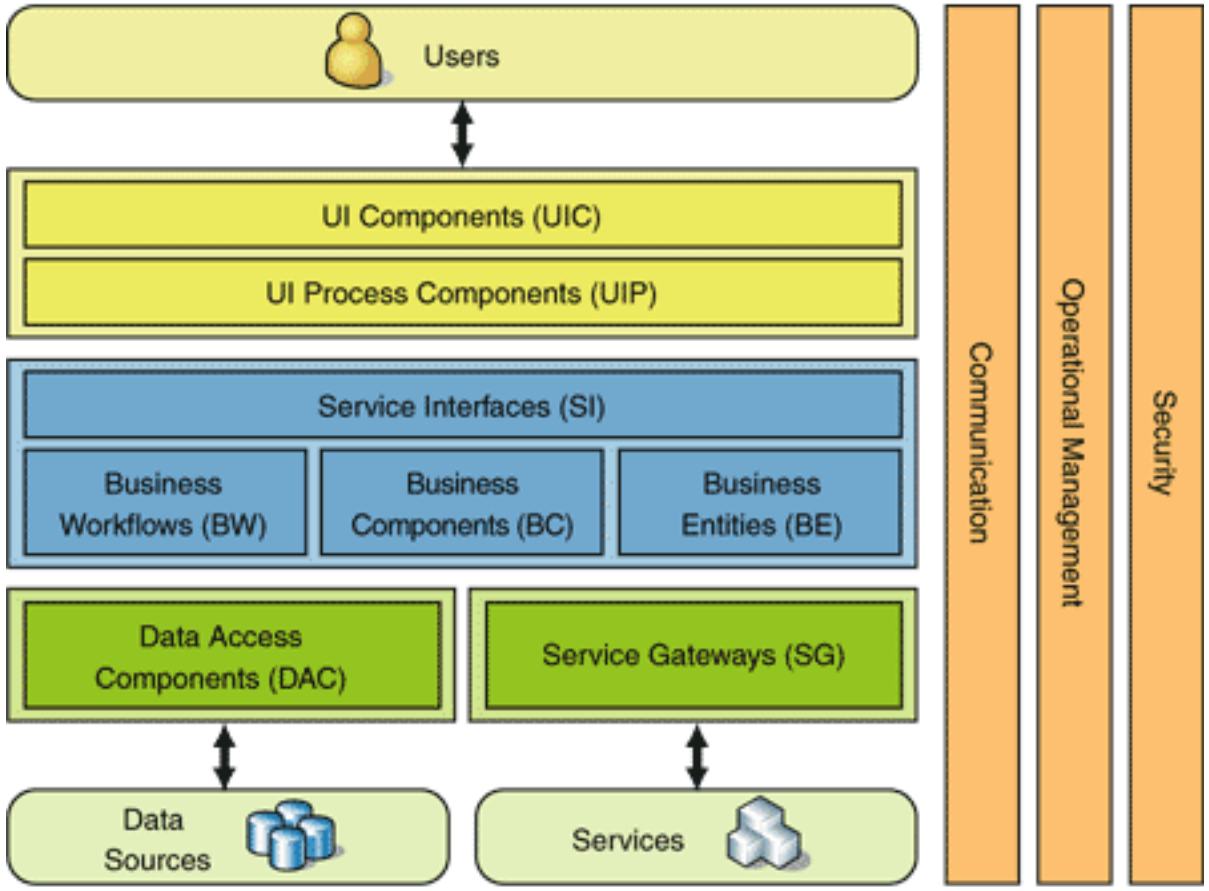
- Some business rules belong in the user interface
  - e.g. initial validation of user input should not require interaction with the business logic/server
- Other business rules should be in the data logic layer
  - e.g. verifying integrity of relationships requires access to other data already in the database
  - Performance gains can result from implementing business rules as stored database procedures

UNIVERSITY of HULL

Three Layer Service Architecture

- Typical business application

- 3-layer design, plus:
  - Expose some functionality as services
  - Consume services provided by other applications



UNIVERSITY of HULL

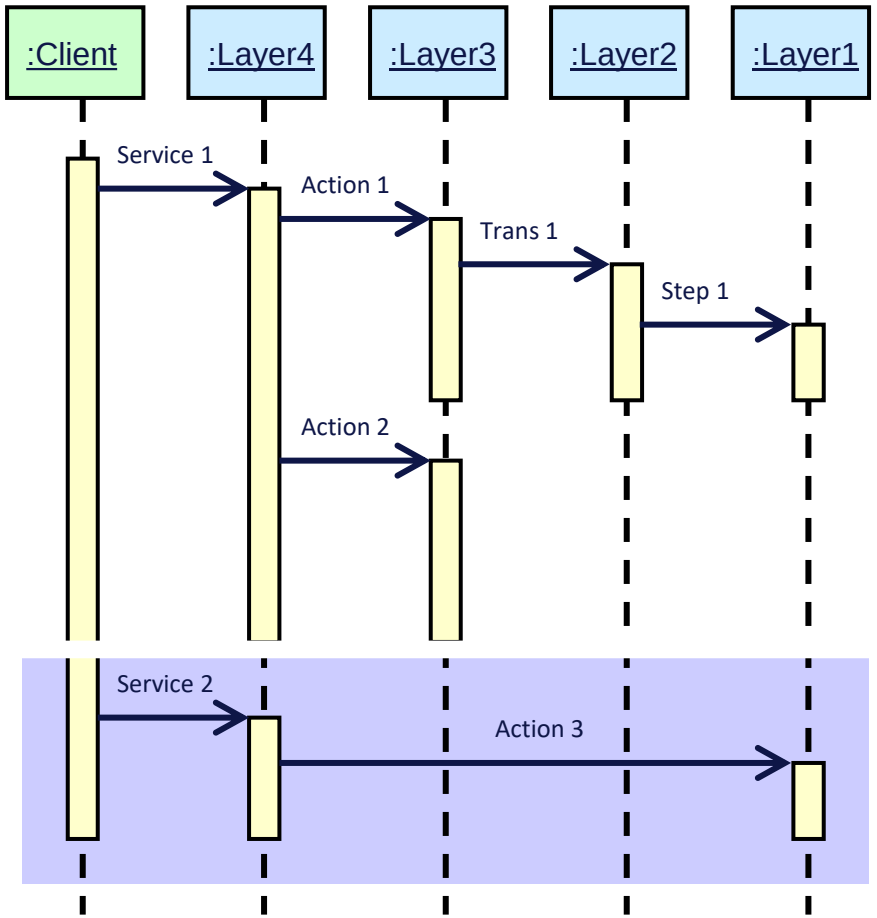
How to make our layers…

- Layering can be achieved in many ways. A few examples…
  - Functions
    - Isolated function which exists on its own
    - May be logically contained in a group of functions which make up one layer
    - Easy to distribute and good for very loose coupling but hard to keep track of
    - Function chains may need to be established within one layer – may be difficult to keep track of but may be easier to maintain/change
    - No central data store (memory) persisted between functions so reliant on data passing and/or dependency injection
  - Classes
    - In-code layers
    - Classes or groups of classes which interact between themselves
    - Each class or group of classes has a specific layer function
    - Often each is logically separated, sometimes by namespace (e.g. myApplication.BusinessLogic, myApplication.DataAccess)
    - Quite easy to accidentally strongly couple these layers so they are dependent on the inner functions of one another
  - Processes
    - Separate processes or groups of processes which exist in isolation
    - Each process or group of processes has a specific layer function
    - Send data between themselves (either on the same machine or networked)
    - Usually much easier to ensure isolation and loose coupling
    - Slower, due to having to send data between components

UNIVERSITY of HULL

Layering Operations

- Strict layering
  - Objects in a layer only depend on (call methods on) peers and those immediately below them
  - Clear, logical layering
  - Easy security boundaries
  - Appropriate for pipelines
- Loose layering
  - Objects in a layer can depend on objects from any lower layer
  - Can be better for efficiency
  - Weaker separation
  - Harder to enforce pipelines/security/filters/etc.

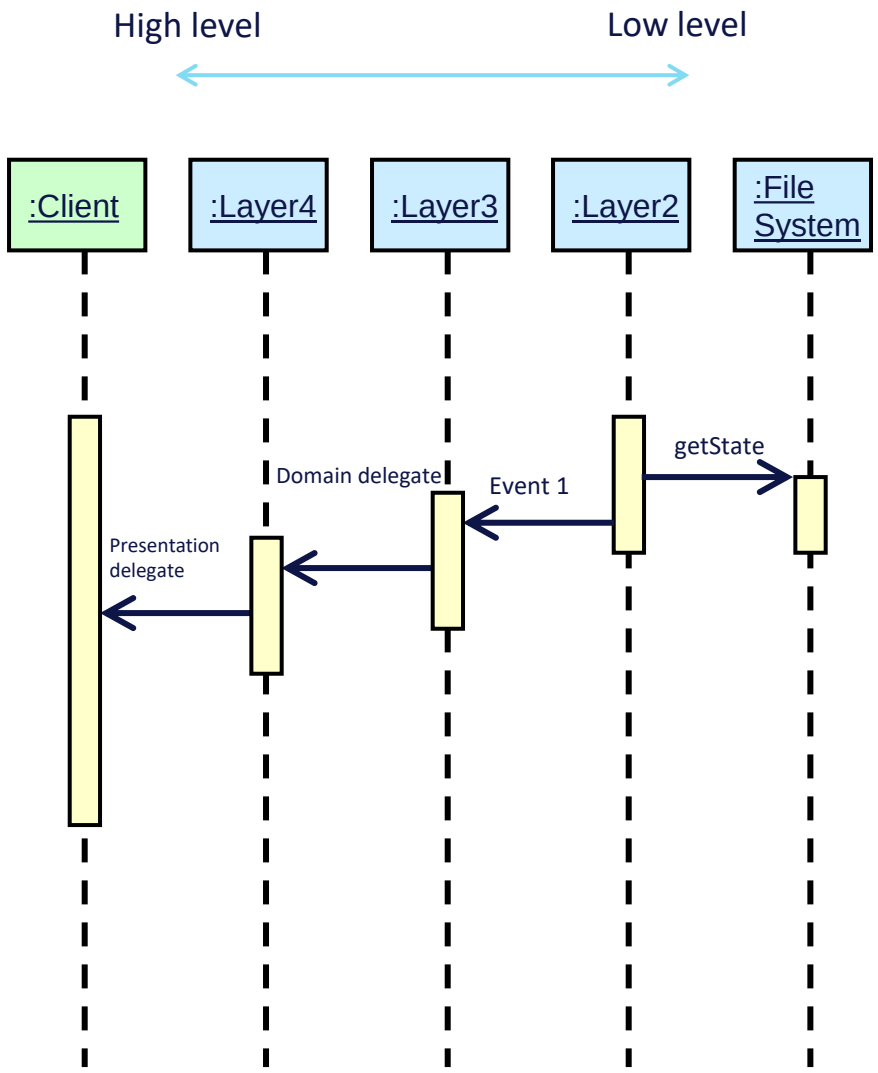


UNIVERSITY of HULL

Upward Layering Operations

- When higher layers make changes to lower layers, they must do so using requests (e.g. calling methods, HTTP Request)

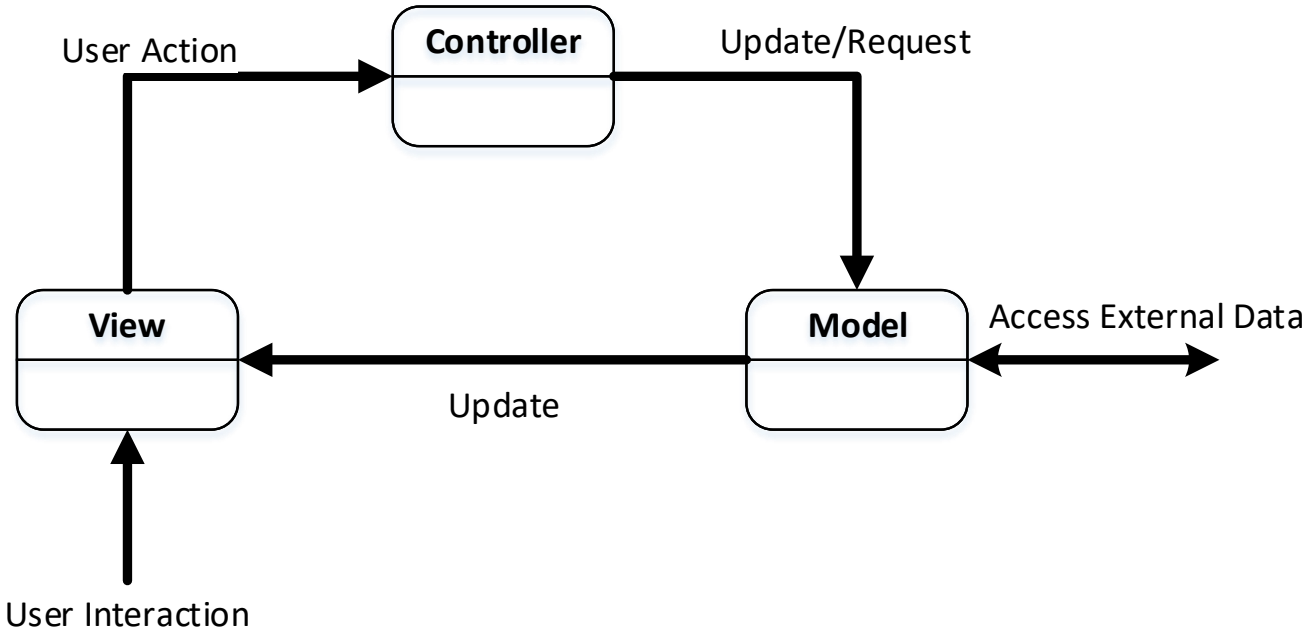
- But higher layers often also need to be informed of changes made at lower levels
  - This is a notification only
  - No dependency!
  - May use delegates (pointer to a method)
    - Or some form of event handler
    - Or higher layers may be dynamically injected



UNIVERSITY of HULL

Common Layered Patterns - MVC

- Three components:
  - **M**odel:
    - Manages Data, Logic and Rules
    - Can be split down into further layers, e.g. Database Access/Data Objects/Business Logic
    - Updates the view
  - **V**iew:
    - An output of information represented for consumption by a user
    - E.g. data converted to a graph for presentation
  - **C**ontroller:
    - Accepts and manages input from the view
    - Converts this input into commands for the model

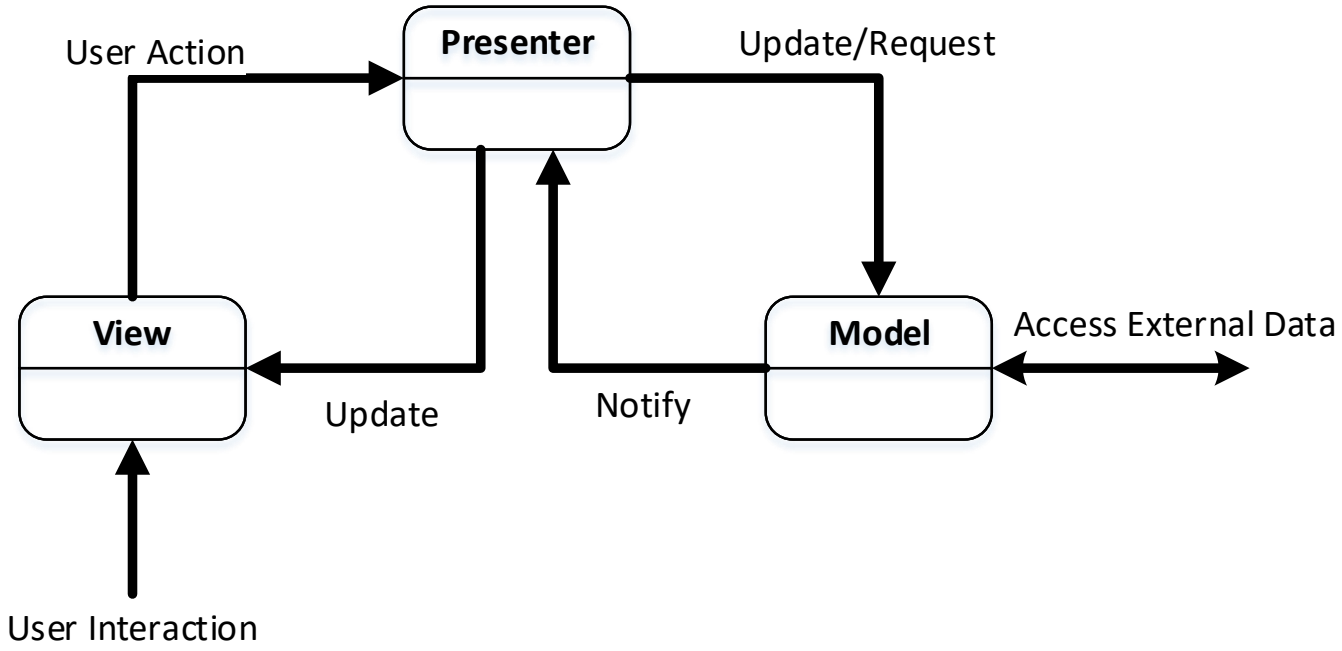


Note: MVC is a pattern and therefore open to interpretation or modification to suit specific requirements

UNIVERSITY of HULL

Common Layered Patterns - MVP

- A variant of MVC
- Three components:
  - **Model:**
    - Manages Data, Logic and Rules
    - Can be split down into further layers, e.g. Database Access/Data Objects/Business Logic
  - **View:**
    - An output of information represented for consumption by a user
    - E.g. data converted to a graph for presentation
  - **Presenter**
    - Accepts and manages input from the view **and model**
    - Converts this input into commands for the model **or view**

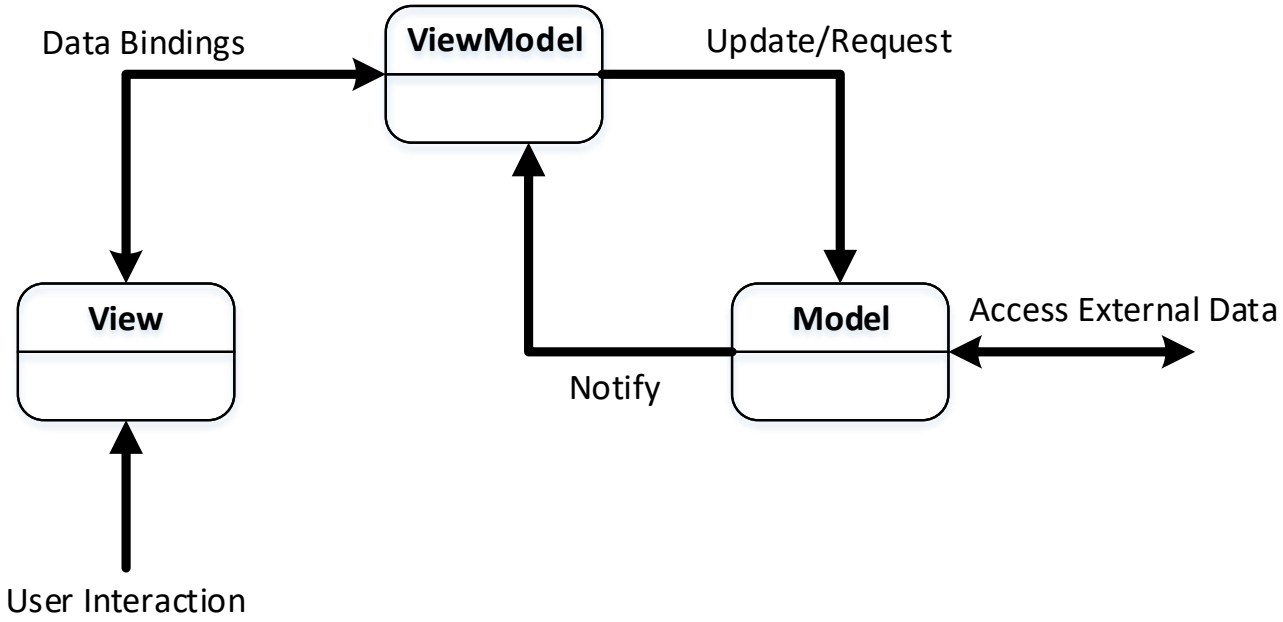


Note: Commonly MVC is implemented as MVP
e.g. ASP.NET MVC is a bit of a mix of MVC/MVP

UNIVERSITY of HULL

Common Layered Patterns - MVVM

- Another variant of MVC
- Three components:
  - **M**odel:
    - Manages Data, Logic and Rules
    - Can be split down into further layers, e.g. Database Access/Data Objects/Business Logic
  - **V**iew:
    - An output of information represented for consumption by a user
    - E.g. data converted to a graph for presentation
  - **V**iew**M**odel
    - Data, behaviours and events are bound directly between the View and the ViewModel
    - The ViewModel and View are explicitly linked by these bindings



MVVM can be used in WPF to bind data in XAML
This allows you to programmatically identify what data to show in the View without being explicit at design time.

Similarly, you can directly bind UI buttons to logic

UNIVERSITY of HULL

More Layers / Fewer Layers

Some applications work well with fewer layers or more layers.

The main considerations are:

- Which elements should exist together to maximise cohesion?
- Which groups of components can exist in isolation?
- What size should the groupings be? Smaller = more maintainable/testable but more expensive boundaries.
- What exists in boundary pipelines? Should these be layers in their own right?

Layers within layers:

- You could consider layers at different hierarchical levels
- A single layer could contain its own layers
  - Possibly led by patterns.

UNIVERSITY of HULL

Common Layered Patterns – Benefits/Problems

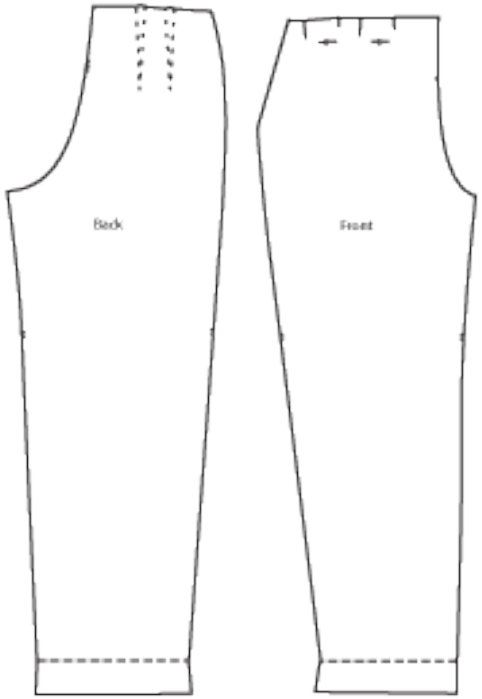
- Faster Development Process
  - Many developer can work simultaneously with little crossover
- Easy to Maintain
  - Parts can be swapped out easily, testing is easier
- Separation of Concerns
  - Data/Business Logic/UI are loosely coupled
- Support for Multiple Views (multiple clients using the same server)
  - Easy to spin up controllers (or similar) for each view whilst using commonly accessible data
- Easy to distribute components on different tiers

But…..

- Increased Complexity
- Boundaries are expensive
- Difficult to change what each layer does later
  - Although, if well made, possible to add/remove layers

UNIVERSITY of HULL

Patterns - Remember



- Patterns are just that = a pattern

- If you make a pair of trousers you use a pattern
- That doesn't mean that you:
  - Use the same sewing machine as everyone else
  - Use the same fabric as everyone else
  - Can't edit/modify/rework the pattern to, for example, add pockets

- It's the same in programming and physical deployment:
  - Patterns are a loose architectural solution to a problem
  - They should help give you an idea of how/why/what
  - The way two companies implement a pattern is likely to be different
  - The way you implement the pattern is up to you
  - The same is true for layers/tiers

UNIVERSITY of HULL

Reading (/viewing)

These links are *old* but still essential reading as patterns don't really change over time because the core problems they solve are still the same – even years later!

(The implementation mechanism(s) do usually change quickly but the problems/patterns don't)

- Patterns:
  - https://msdn.microsoft.com/en-us/library/ee658117.aspx
- Layered Apps:
  - https://msdn.microsoft.com/en-gb/library/ee658109.aspx
- Tiered Systems:
  - https://msdn.microsoft.com/en-us/library/ee658120.aspx

Reflection

In this session we've looked at:

- Layered Software Architectures for loose application coupling and flexible application design
- Tiered Hardware Architectures for physical loose coupling with boundaries, optimised hardware and separation of concerns
- Model View Controller (MVC)
- Model View ViewModel (MVVM)
- Model View Presenter (MVP)

In the next session we will continue to look at Design and Deployment: Common Distributed Service Types and Centralised/Decentralised Architectures

UNIVERSITY of HULL